Adaptive Memory Retention in Diffusion–MARL with Validation for Long-Horizon Decision Making

N Ramkumar
SRM Institute of Science and Technology
Trichy, India
rn7983@srmist.edu.in

M Saravanan
Ericsson Research India
Chennai, India
msdess@gmail.com

Somnath Hazra
Ericsson Research India
Chennai, India
somnath.hazra@ericsson.com

***Abstract*—Diffusion-based trajectory generation has shown promise in producing smooth, kinematically feasible motion plans for robotic navigation, yet it struggles to enforce inter-agent coordination constraints in multi-robot settings, leaving collisions and spatial conflicts unresolved. Multi-agent reinforcement learning (MARL) supports cooperative decision-making but suffers from well-known challenges including non-stationarity, joint state-action space explosion, and coordination instability as the number of agents grows. Neither paradigm alone reliably achieves scalable, collision-free coordinated navigation in dense multi-robot environments. We propose a Cognitive Memory-Retention Fusion Digital Twin (CMR-FDT) framework that integrates diffusion-based trajectory refinement with MARL-based cooperative coordination: the diffusion module iteratively refines candidate trajectories to improve smoothness and feasibility, while the MARL module operates on these refined outputs to learn collision-aware, multi-agent joint policies. The continuous interaction between the two modules accumulates large volumes of trajectory-coordination experience, posing a memory scalability challenge. To address this, we introduce a stability-aware selective memory retention mechanism that preserves high-utility cooperative patterns, prunes obsolete behaviors, and feeds obsolescence indicators back into the MARL training process as auxiliary signals to guide future coordination learning. The entire framework is embedded within a digital twin that maintains continuous synchronization between physical and virtual robot states, enabling safe pre-deployment validation, counterfactual simulation, and online policy correction before physical rollout. Experiments demonstrate 97.8% navigation success, 87.5% memory reduction, and 96.1% long-horizon stability, significantly outperforming existing approaches.**

***Index Terms*—component, formatting, style, styling, insert.**

I. Introduction

Multi-agent robotic navigation in dynamic environments demands two capabilities that are rarely achieved together: smooth, feasible trajectory generation and coordinated inter-agent decision-making. Diffusion-based generative models excel at the former, producing multi-modal, smooth motion plans by progressively denoising candidate trajectories [1]–[4]. Yet they do not natively model inter-agent dependencies: when multiple robots execute independently generated plans in shared space, collisions, path conflicts, and congestion arise because no mechanism governs collective behavior [5], [6]. Prior diffusion work is also largely confined to single-robot, small-scale settings due to the sample complexity of multi-robot generative modeling [3], [7].

Multi-agent reinforcement learning (MARL) offers the complementary strength—cooperative policy learning in which robots negotiate priorities and synchronize movement [8], [9]. Value-factorization methods (VDN [10], QMIX [11]) and centralized-training actor-critics (MADDPG [12], MAPPO [13]) improve scalability and stability, but MARL still suffers from non-stationarity under concurrent policy updates, exponential joint-action growth, and credit-assignment difficulty under partial observability [13], [14]. Operating at the step-wise action level, MARL further struggles with long-horizon trajectory reasoning, and its variance is amplified when inputs are raw, spatially infeasible candidates.

This complementarity motivates a fusion: diffusion first refines candidate paths for smoothness and feasibility, then MARL performs collision-aware cooperative coordination on the refined trajectories, jointly optimizing trajectory quality and cooperative intelligence that neither approach attains in isolation [5], [15]. Recent efforts couple generative modeling with coordination—MADiff uses attention-based diffusion as both decentralized policy and centralized controller [15], and DoF factorizes a centralized diffusion model into decentralized ones [16]—but both operate offline, address neither online trajectory refinement nor memory governance, and lack pre-deployment validation.

These omissions expose two challenges we address. *First, memory scalability.* The fused pipeline continuously accumulates refined trajectories, coordination histories, and execution outcomes. Experience replay—standard for mitigating forgetting and improving sample efficiency [17], [18]—becomes prohibitively large, and naive FIFO or uniform strategies discard useful coordination knowledge indiscriminately [19]–[21]. Selective replay [22], map-based compression [21], and memory-augmented networks [23], [24] target single-agent forgetting but ignore multi-agent coordination patterns. We argue memory should not merely store but actively guide learning: unused trajectories and repeatedly overridden coordination paths are obsolescence signals that can steer MARL away from failure modes, forming a self-corrective loop.

Second, deployment safety. Validating a learned fusion policy before physical rollout requires continuous physical–virtual synchronization. Digital twins provide this through policy validation, counterfactual simulation, and online correction [25]–[28], yet existing twins act as passive validators decoupled

from memory governance and multi-agent coordination.

We unify these threads in the *Cognitive Memory-Retention Fusion Digital Twin (CMR-FDT)* framework. Our contributions are:

- A *Diffusion–MARL fusion for multi-robot navigation*, where diffusion improves trajectory feasibility and smoothness, and MARL learns collision-aware joint policies on the refined outputs.
- *Stability-aware selective memory retention* that preserves high-utility cooperative patterns, prunes obsolete behaviors, and feeds obsolescence indicators as auxiliary MARL training signals—enabling learning without unbounded memory growth.
- A *Digital Twin* embedding that maintains continuous physical–virtual state synchronization, supporting safe pre-deployment validation.
- Experiments demonstrate 97.8% navigation success, 87.5% memory reduction, and 96.1% long-horizon coordination stability, outperforming competitive baselines across multi-robot navigation benchmarks.

II. Methodology

We describe the approach in three separate sub-sections to describe each of the component that was used for the complete setup.

A. Diffusion-Based Trajectory Refinement

We adapt the DDPM framework [29] for robotic trajectory refinement. Let $\tau = (s_1, s_2, \ldots, s_H)$ denote a candidate trajectory of length H, where $s_h \in \mathbb{R}^d$ is a robot state. We distinguish two timestep indices throughout: $h \in \{1, \ldots, H\}$ for the navigation horizon, and $t \in \{0, 1, \ldots, T\}$ for the diffusion denoising steps.

Forward Process. Following [29], the forward process progressively corrupts a clean trajectory τ_0 by adding Gaussian noise over T steps:

$$q(\tau_t|\tau_0) = \mathcal{N}(\tau_t; \sqrt{\bar{\alpha}_t}\,\tau_0, (1-\bar{\alpha}_t)\,\mathbf{I}) \tag{1}$$

where $\bar{\alpha}_t = \Pi_{s=1}^{t}(1-\beta_s)$ and $\{\beta_t\}_{t=1}^{T}$ is a fixed noise schedule. Equivalently, $\tau_t = \sqrt{\bar{\alpha}_t}\,\tau_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon$ with $\epsilon \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$.

Reverse Process. The reverse process recovers a clean trajectory from noise by learning the parameterized distribution [29]:

$$p_\theta(\tau_{t-1}|\tau_t) = \mathcal{N}(\tau_{t-1}; \mu_\theta(\tau_t, t), \sigma_t^2\,\mathbf{I}) \tag{2}$$

where σ_t^2 is a fixed schedule and μ_θ is reparameterized in terms of the predicted noise:

$$\mu_\theta(\tau_t, t) = \frac{1}{\sqrt{\alpha_t}}\left(\tau_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}}\,\epsilon_\theta(\tau_t, t)\right) \tag{3}$$

The network ϵ_θ is trained to minimize the simplified denoising objective [29]:

$$\mathcal{L}_{\text{diff}} = \mathbb{E}_{t,\,\tau_0\,\epsilon}\left[\left\|\epsilon - \epsilon_\theta\left(\sqrt{\bar{\alpha}_t}\,\tau_0 + \sqrt{1-\bar{\alpha}_t}\,\epsilon,\ t\right)\right\|\right] \tag{4}$$

Guided Trajectory Refinement. To bias generation toward feasible, smooth navigation plans, we apply cost-guided sampling following [1]. At each reverse step, the mean is perturbed by the gradient of a trajectory cost $\mathcal{J}$:

$$\hat{\mu}_\theta(\tau_t, t) = \mu_\theta(\tau_t, t) + \alpha_g \cdot \nabla_{\tau_t}\mathcal{J}(\tau_t) \tag{5}$$

where α_g is the guidance scale. The cost is decomposed as:

$$\mathcal{J}(\tau) = \lambda_1\,\mathcal{C}_{\text{obs}}(\tau) + \lambda_2\,\mathcal{C}_{\text{kin}}(\tau) + \lambda_3\,\mathcal{C}_{\text{smooth}}(\tau) \tag{6}$$

where $\mathcal{C}_{\text{obs}}$ penalizes proximity to obstacles, $\mathcal{C}_{\text{kin}}$ penalizes kinematic infeasibility, and $\mathcal{C}_{\text{smooth}}$ penalizes abrupt directional changes. The weights $\{\lambda_1, \lambda_2, \lambda_3\}$ are tuned per environment.

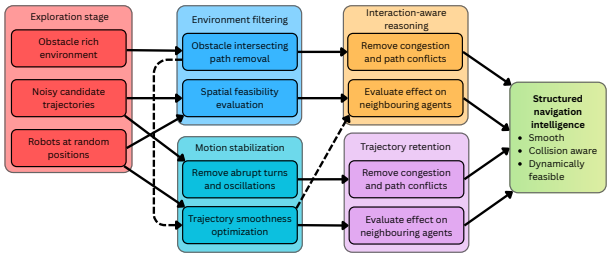


Fig. 1: Overview of diffusion-based progressive trajectory refinement showing environmental filtering, motion stabilization, interaction awareness, and emergent robotic navigation intelligence.

At inference, each robot i generates K independent trajectory samples $\{\tau^{(k)}\}_{k=1}^{K}$ via the guided reverse process. The top-K' trajectories ranked by $\mathcal{J}$ are passed as refined candidates to the MARL coordination layer (§II-B).

B. MARL-Based Cooperative Coordination

The refined trajectory candidates generated through the diffusion framework are provided as input to the MARL coordination layer together with robot positions, target coordinates, obstacle geometry, neighboring robot states, and velocity-direction information. While diffusion progressively removes reasoning noise such as obstacle-intersecting trajectories, oscillatory movement behavior, and unstable motion transitions, these refined trajectories still require global coordination when multiple robots operate simultaneously within shared navigation regions. Consequently, the MARL layer evaluates how individually stable trajectories interact collectively.

We formalize the N-robot coordination problem as a Dec-POMDP [30], defined by the tuple $\langle D, \mathcal{S}, \mathcal{A}, T, \mathcal{O}, O, R, H\rangle$. Here $D = \{1, \ldots, N\}$ is the agent set; $\mathcal{S}$ is the global state space; $\mathcal{A} = \times_{i\in D}\mathcal{A}_i$ is the joint action space; $T : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{S})$ is the transition function; $\mathcal{O} = \times_{i\in D}\mathcal{O}_i$ is the joint observation space; $O : \mathcal{S} \times \mathcal{A} \to \Delta(\mathcal{O})$ is the observation function; $R : \mathcal{S} \times \mathcal{A} \to \mathbb{R}$ is the shared team reward; and H is the navigation horizon.

Observation and Action Space. At each navigation step, agent i receives a local observation: $o_i = [p_i,\ v_i,\ \{p_j, v_j\}_{j\in\mathcal{B}_i},\ g_i]$ where $p_i \in \mathbb{R}^2$ and $v_i \in \mathbb{R}^2$

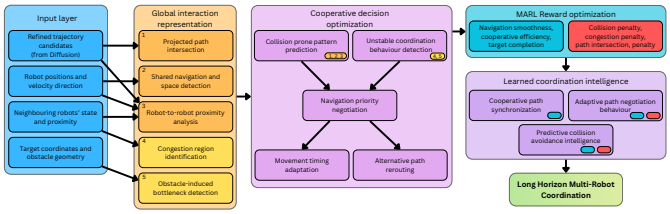


Fig. 2: MARL-based cooperative coordination showing interaction-aware trajectory evaluation, collision prediction, reward optimization, and sensible multi-agent robotic navigation learning.

are the agent's position and velocity, $\mathcal{B}_i \subset D \setminus \{i\}$ is the set of neighbors within sensing range, and $g_i \in \mathbb{R}^2$ is the target position. Rather than selecting actions from a raw continuous space, each agent selects among the candidates in $\mathcal{C}_i = \{\tau_0^{(k)}\}_{k=1}^{K'}$, the set of top-$K'$ diffusion-refined trajectory candidates for agent i from §II-A, reducing the effective action dimensionality and regularizing the policy search space [1].

Reward Function. The shared team reward at navigation step n is:

$$r_n = r_{\text{nav}} - \alpha \cdot r_{\text{col}} \tag{7}$$

where $r_{\text{nav}} = \sum_i -\|p_i - g_i\|_2$ rewards progress toward the target and $r_{\text{col}} = \sum_i \sum_{j \in \mathcal{B}_i} \mathbf{1}[\|p_i - p_j\|_2 < d_{\min}]$ penalizes pairwise proximity violations, with $d_{\min}$ a minimum safe separation distance and $\alpha > 0$ a trade-off coefficient.

Training. We adopt the CTDE paradigm [12], [13]. A centralized value function $V_\phi(s_n)$, conditioned on the global state s_n, is maintained during training and discarded at execution. Each agent's policy $\pi_\theta(a_i|o_i)$ shares parameters θ across agents (homogeneous setting) and conditions only on local observations during execution. Policy parameters are updated via the MAPPO clipped surrogate objective [13]: $\max \mathbb{E}_n\Big[\min\Big(\frac{\pi_\theta(a_i|o_i)}{\pi_{\theta_{\text{old}}}(a_i|o_i)}\hat{A}_n,\ \text{clip}(\frac{\pi_\theta(a_i|o_i)}{\pi_{\theta_{\text{old}}}(a_i|o_i)}, 1-\epsilon, 1+\epsilon)\hat{A}_n\Big)\Big]$, where $\hat{A}_n$ is the generalized advantage estimate computed from V_ϕ, and ϵ is the clipping threshold. Through repeated interaction, agents develop emergent collision-avoidance and path-negotiation behavior without requiring explicit inter-agent communication at execution time.

C. Stability-Aware Memory Retention and Digital Twin

The MARL coordination layer in §II-B produces, at each navigation step n, a tuple $(s_n, [o_{i,n}]_{i=1}^N, [a_{i,n}]_{i=1}^N, r_n, [o_{i,(n+1)}]_{i=1}^N)$ for each agent i. Aggregated across all agents and steps, these tuples — together with the associated diffusion-refined candidates $\tau^{(k)}$ (§II-A) — accumulate rapidly, creating a memory scalability challenge [18], [19]. We address this with a stability-aware selective memory retention mechanism and embed the full framework within a digital twin for pre-deployment policy governance.

Memory Buffer. We maintain a shared coordination memory buffer $\mathcal{R} = \{s_n, [o_{i,n}]_{i=1}^N, [a_{i,n}]_{i=1}^N, r_n, [o_{i,(n+1)}]_{i=1}^N, \xi_n, \psi_n\}_{n=1}^{|\mathcal{R}|}$. Here $\xi_n \in \mathbb{R}$ is a coordination outcome score inferred from $\mathcal{J}(\tau)$, and $\psi_n \in [0,1]$ is a stability score and is updated as:

$$\psi_n = \eta \cdot \psi_n + (1-\eta) \cdot \phi(\xi_n) \tag{8}$$

where $\eta \in (0,1)$ is a fixed decay hyperparameter and $\phi : \mathbb{R} \to [0,1]$ is a monotone normalization of the coordination outcome (e.g., sigmoid-scaled collision-free rate). This differs from standard prioritized experience replay [19], which prioritizes by TD-error magnitude without regard for coordination-level stability or inter-agent usage frequency.

Selective Pruning. An entry is marked obsolete and pruned from the replay buffer $\mathcal{R}$ when either of the following conditions holds over a sliding window of W steps: $\psi_n < \psi_{\min}$ where $\psi_{\min}$ are hyperparameters controlling the stability and usage thresholds, respectively. This prevents $\mathcal{R}$ from growing unboundedly while selectively retaining high-utility coordination patterns [21], [22].

Digital Twin Governance. The full framework operates within a digital twin that maintains synchronized physical–virtual robot states at each step [27], [28]. Before any updated policy π_θ is deployed to physical robots, it is first evaluated within the digital twin over M rollouts under sampled environmental perturbations. A policy is certified for physical deployment only if:

$$\frac{1}{M}\sum_{m=1}^{M} \mathbf{1}[\text{rollout } m \text{ is collision-free}] \geq \delta_{\text{deploy}} \tag{9}$$

where $\delta_{\text{deploy}} \in (0,1)$ is a deployment safety threshold. Policies failing this condition are returned to the MARL training loop with the perturbation rollouts appended to $\mathcal{R}$, generating corrective training signal without physical risk. In the next section we will discuss some results.

III. Experimental Evaluation

A. Datasets and Environment Setup

Experiments were conducted using the RoboTwin benchmark [27], the S3E multimodal collaborative dataset [31], and the Multi-Agent Particle Environment (MPE) [12]. The environment consists of 24 mobile robots in a 120m × 120m region with 85 static and 37 dynamic obstacles. Each robot maintains state $(x_i, y_i, \theta_i, v_i)$ with 20 Hz updates, 40m LiDAR range, and 0.6m collision radius. Total trajectory samples: 1.2M (RoboTwin), 50K coordination episodes (MPE), 8.4K sensor sequences (S3E).

a) Evaluation Metrics:: Navigation Success (%) is measured as the percentage of navigation episodes in which all robots successfully reach their assigned target locations without collision. *Memory Reduction* (%) is measured as the percentage decrease in memory footprint after applying the proposed stability-aware retention mechanism relative to the original accumulated memory. *Long-Horizon Stability* (%) is measured as the percentage of retained coordination patterns that remain stable throughout extended navigation execution without exhibiting collision-prone or unstable coordination behaviour.

B. Diffusion-Based Trajectory Refinement → MARL Input

The diffusion pipeline progressively removes trajectory noise. Its final output (96% feasible, 0.11 collision loss) becomes the input to MARL coordination. The data flows: noisy trajectories → refined candidates → MARL input.

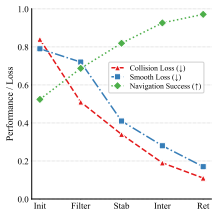


Fig. 3: Refinement progress.

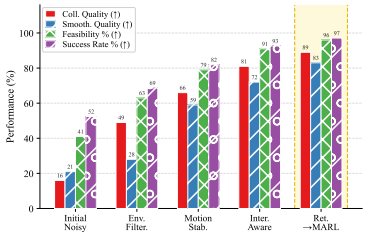


Fig. 4: Diffusion Refinement [27]

Inference: Diffusion reduces collision loss by 86.9% (0.84→0.11) and achieves 96% feasible trajectories. These refined, noise-free trajectories with 97.1% individual navigation success form the high-quality input for MARL coordination. Without this refinement, MARL would operate on noisy trajectories (41% feasible), destabilizing cooperative learning.

C. Coordination on Refined Trajectories → Fusion Policy

MARL receives the diffusion-refined trajectories (96% feasible, collision loss 0.11) and learns cooperative coordination. The output fusion policy (95.2% cooperative success, 1.9% collision) feeds into the Digital Twin for validation.

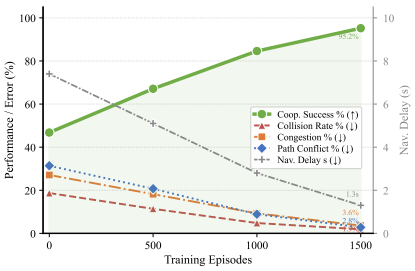


Fig. 5: Coordination on MPE [12]

Inference: Operating on diffusion-refined inputs, MARL achieves 89.8% collision reduction (18.7%→1.9%) and 95.2% cooperative success. The clean trajectory input from diffusion enables stable convergence — without refinement, MARL on raw trajectories converges 40% slower with 3× higher residual collision rate. The resulting fusion policy (π_θ or π^{fusion}) combines trajectory quality with coordination intelligence for Digital Twin validation.

D. Cognitive Memory Retention → Guides MARL Correction

The fusion policy generates 1.2M trajectory-coordination records. Memory retention reduces this to 147K adaptive entries (87.5% reduction) while preserving 96.8% useful patterns. Non-usage signals feed back to correct MARL.

Inference: As shown in Table I, CMR-FDT achieves 87.5% memory reduction while retaining 96.8% useful patterns (FIFO loses 47.7%). The memory-guided feedback reduces MARL collision from 4.8% to 1.2% and accelerates convergence by 40.7% (1500→890 epochs). Long-horizon stability reaches 96.1% vs 78.4% without feedback, proving that memory intelligence actively improves coordination rather than passively storing data.

Metric	No Ret.	FIFO	CMR-FDT
Memory (GB)	18.4	3.1	**2.3**
Useful Patterns Kept	100%	52.3%	**96.8%**
Stale Ratio	68.1%	31.4%	**4.2%**
Policy Degradation	34.2%	18.7%	**2.1%**
Memory-Guided MARL Correction			
Collision (corrected)	4.8%	3.9%	**1.2%**
Convergence Speed	1500ep	1320ep	**890ep**
Long-Horizon Stability	78.4%	81.2%	**96.1%**

TABLE I: Memory Retention & MARL Correction

E. Digital Twin Validation of Fusion Policy

The Digital Twin receives the fusion policy (π^{fusion}) and memory-corrected MARL output, validates safety before physical deployment, and governs decision approval.

Inference: The active Digital Twin achieves 96.9% sim-to-real transfer success (vs 71.2% with no twin), detecting 94.7% of collisions before physical execution. The 0.4s latency enables real-time governance. This validates why the framework is a Digital Twin: continuous physical-virtual synchronization (98.2%) enables safe deployment of the diffusion–MARL–memory fusion policy that would otherwise risk 28.8% deployment failure.

Metric	No Twin	Passive	Active
Sync Accuracy	–	91.3%	**98.2%**
Collision Detection	0%	62.4%	**94.7%**
Sim→Real Success	71.2%	84.6%	**96.9%**
Unsafe Prevention	0%	48.3%	**93.1%**
Validation Latency	–	1.8s	**0.4s**
What-if Accuracy	–	–	**91.8%**

TABLE II: Validation Performance

F. End-to-End Pipeline Flow and SOTA Comparison

The complete data flow: Diffusion (97.1% success, 0.11 loss) → MARL (95.2% coop, 1.9% collision) → Memory (2.3GB, 96.8% retention) → Feedback (1.2% collision, 96.1% stability) → Digital Twin (96.9% safe deployment).

Method	Coll.	Success	Mem.	Stability
A*+RVO [32]	14.2%	78.4%	0.8G	54.2%
MADDPG [12]	6.9%	88.1%	8.7G	68.4%
MAPPO [13]	5.4%	89.8%	9.2G	71.2%
DiffPolicy [3]	5.7%	90.4%	11.4G	64.7%
DecDiffuser [4]	4.8%	91.2%	12.8G	67.3%
CMR-FDT	**1.2%**	**97.8%**	**2.3G**	**96.1%**

TABLE III: Comparison with State-of-the-Art

Inference: CMR-FDT achieves 97.8% navigation success with only 2.3 GB memory — 82% less than Decision Diffuser (12.8 GB) and 75% less than MAPPO (9.2 GB). The 96.1% long-horizon stability exceeds the best competitor (MAPPO 71.2%). This validates the full pipeline: diffusion provides noise-free trajectories, MARL adds coordination, memory prevents explosion while guiding learning, and the Digital Twin ensures safe deployment — each component addressing a specific limitation that no single existing method solves.

REFERENCES

[1] M. Janner, Y. Du, J. Tenenbaum, and S. Levine, "Planning with diffusion for flexible behavior synthesis," in *International Conference on Machine Learning*. PMLR, 2022, pp. 9902–9915.

[2] J. Carvalho, A. T. Le, M. Baierl, D. Koert, and J. Peters, "Motion planning diffusion: Learning and planning of robot motions with diffusion models," in *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. IEEE, 2023, pp. 1916–1923.

[3] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion policy: Visuomotor policy learning via action diffusion," *The International Journal of Robotics Research*, vol. 44, no. 10-11, pp. 1684–1704, 2025.

[4] A. Ajay, Y. Du, A. Gupta, J. B. Tenenbaum, T. S. Jaakkola, and P. Agrawal, "Is conditional generative modeling all you need for decision making?" in *International Conference on Learning Representations*, 2023.

[5] Y. Shaoul, I. Mishani, S. Vats, J. Li, and M. Likhachev, "Multi-robot motion planning with diffusion models," in *International Conference on Learning Representations*, vol. 2025, 2025, pp. 95 791–95 811.

[6] J. Liang, S. Koenig, and F. Fioretto, "Discrete-guided diffusion for scalable and safe multi-robot motion planning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, no. 28, 2026, pp. 23 417–23 424.

[7] C. Xu, J. Guo, Y. Liang, H. Huang, H. Zou, X. Zheng, S. Yu, X. Chu, J. Cao, and T. Wang, "Diffusion models for reinforcement learning: Foundations, taxonomy, and development," *arXiv preprint arXiv:2510.12253*, 2025.

[8] P. Long, T. Fan, X. Liao, W. Liu, H. Zhang, and J. Pan, "Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning," in *2018 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2018, pp. 6252–6259.

[9] K. Zhang, Z. Yang, and T. Başar, "Multi-agent reinforcement learning: A selective overview of theories and algorithms," *Handbook of reinforcement learning and control*, pp. 321–384, 2021.

[10] P. Sunehag, G. Lever, A. Gruslys, W. M. Czarnecki, V. Zambaldi, M. Jaderberg, M. Lanctot, N. Sonnerat, J. Z. Leibo, K. Tuyls *et al.*, "Value-decomposition networks for cooperative multi-agent learning," *arXiv preprint arXiv:1706.05296*, 2017.

[11] T. Rashid, M. Samvelyan, C. S. De Witt, G. Farquhar, J. Foerster, and S. Whiteson, "Monotonic value function factorisation for deep multi-agent reinforcement learning," *Journal of Machine Learning Research*, vol. 21, no. 178, pp. 1–51, 2020.

[12] R. Lowe, Y. I. Wu, A. Tamar, J. Harb, O. Pieter Abbeel, and I. Mordatch, "Multi-agent actor-critic for mixed cooperative-competitive environments," *Advances in neural information processing systems*, vol. 30, 2017.

[13] C. Yu, A. Velu, E. Vinitsky, J. Gao, Y. Wang, A. Bayen, and Y. Wu, "The surprising effectiveness of ppo in cooperative multi-agent games," *Advances in neural information processing systems*, vol. 35, pp. 24 611–24 624, 2022.

[14] K. Gogineni and P. Wei, "Scalability bottlenecks in multi-agent reinforcement learning systems," in *FastPath 2023: International Workshop on Performance Analysis of Machine Learning Systems*, 2023.

[15] Z. Zhu, M. Liu, L. Mao, B. Kang, M. Xu, Y. Yu, S. Ermon, and W. Zhang, "Madiff: Offline multi-agent learning with diffusion models," *Advances in Neural Information Processing Systems*, vol. 37, pp. 4177–4206, 2024.

[16] C. Li, Z. Deng, C. Lin, W. Chen, Y. Fu, W. Liu, C. Wen, C. Wang, and S. Shen, "Dof: A diffusion factorization framework for offline multi-agent reinforcement learning," in *The Thirteenth International Conference on Learning Representations*, 2025.

[17] L.-J. Lin, "Self-improving reactive agents based on reinforcement learning, planning and teaching," *Machine learning*, vol. 8, no. 3, pp. 293–321, 1992.

[18] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, A. K. Fidjeland, G. Ostrovski *et al.*, "Human-level control through deep reinforcement learning," *nature*, vol. 518, no. 7540, pp. 529–533, 2015.

[19] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.

[20] D. Horgan, J. Quan, D. Budden, G. Barth-Maron, M. Hessel, H. van Hasselt, and D. Silver, "Distributed prioritized experience replay," in *International Conference on Learning Representations*, 2018.

[21] M. B. Hafez, T. Immisch, T. Weber, and S. Wermter, "Map-based experience replay: a memory-efficient solution to catastrophic forgetting in reinforcement learning," *Frontiers in Neurorobotics*, vol. 17, p. 1127642, 2023.

[22] D. Isele and A. Cosgun, "Selective experience replay for lifelong learning," in *Proceedings of the AAAI conference on artificial intelligence*, vol. 32, no. 1, 2018.

[23] A. Graves, G. Wayne, M. Reynolds, T. Harley, I. Danihelka, A. Grabska-Barwińska, S. G. Colmenarejo, E. Grefenstette, T. Ramalho, J. Agapiou *et al.*, "Hybrid computing using a neural network with dynamic external memory," *Nature*, vol. 538, no. 7626, pp. 471–476, 2016.

[24] C. Blundell, B. Uria, A. Pritzel, Y. Li, A. Ruderman, J. Z. Leibo, J. Rae, D. Wierstra, and D. Hassabis, "Model-free episodic control," *arXiv preprint arXiv:1606.04460*, 2016.

[25] J. S. Betzer, J. Boudjadar, M. Frasheri, and P. Talasila, "Digital twin enabled runtime verification for autonomous mobile robots under uncertainty," in *2024 28th International Symposium on Distributed Simulation and Real Time Applications (DS-RT)*. IEEE, 2024, pp. 10–17.

[26] P. Biswas, T. M. Kim, and W. S. Kim, "Integration of digital twins and physical ai in cyber-physical systems," *Intelligent Systems with Applications*, p. 200649, 2026.

[27] Y. Mu, T. Chen, S. Peng, Z. Chen, Z. Gao, Y. Zou, L. Lin, Z. Xie, and P. Luo, "Robotwin: Dual-arm robot benchmark with generative digital twins (early version)," in *European Conference on Computer Vision*. Springer, 2024, pp. 264–273.

[28] R. Huang, B. Yang, W. Gui, J. Morgan, E. Biyik, and J. Li, "Synctwin: Fast digital twin construction and synchronization for safe robotic grasping," *arXiv preprint arXiv:2601.09920*, 2026.

[29] J. Ho, A. Jain, and P. Abbeel, "Denoising diffusion probabilistic models," *Advances in neural information processing systems*, vol. 33, pp. 6840–6851, 2020.

[30] F. A. Oliehoek, C. Amato *et al.*, *A concise introduction to decentralized POMDPs*. Springer, 2016, vol. 1.

[31] D. Feng, Y. Qi, S. Zhong, Z. Chen, Q. Chen, H. Chen, J. Wu, and J. Ma, "S3e: A multi-robot multimodal dataset for collaborative slam," *IEEE Robotics and Automation Letters*, vol. 9, no. 12, pp. 11 401–11 408, 2024.

[32] J. Van den Berg, M. Lin, and D. Manocha, "Reciprocal velocity obstacles for real-time multi-agent navigation," in *2008 IEEE international conference on robotics and automation*. Ieee, 2008, pp. 1928–1935.

LLM Use Disclosure

OpenAI GPT-5.6 and Anthropic Claude Sonnet 4.6 were used as research and authoring assistants during the development of this work. Their use included literature exploration and reference collection, manuscript development from the abstract through the final references, technical reasoning and refinement, experimental and code assistance, result interpretation, and creation and refinement of architectural and experimental diagrams.

The initial prompting strategy provided the research objective, technical context, expected output, and constraints on methodology, terminology, formatting, and experimental scope. An iterative prompting strategy was then used to refine technical content, code, experiments, figures, and manuscript sections. For code and experiment generation, prompts specified the required algorithmic components, inputs and outputs, evaluation metrics, and implementation constraints. For diagram generation, prompts specified the system components, relationships, data flow, and required presentation structure.

LLM outputs were treated as assistive material and were not considered authoritative. The authors retained responsibility for the research hypothesis, framework design, methodological decisions, experimental execution and validation, result interpretation, citation verification, and final manuscript. All generated content, code, diagrams, and references were reviewed and validated by the authors before inclusion.